

UNIVERSIDAD NACIONAL DE INGENIERÍA (UNI) RECINTO UNIVERSITARIO SIMÓN BOLÍVAR (RUSB) ÁREA DEL CONOCIMIENTO TECNOLOGIA DE LA INFORMACION Y LA COMUNICACION	
ASIGNATURA:	Diseño de Sistemas en Internet
DOCENTE:	M.Sc. Walter José Quintero Acevedo
PERIODO:	II Semestre 2026

GUÍA ACADÉMICA: BACKEND, SERVICIOS WEB Y ARQUITECTURA DE APIS

PRESENTACIÓN GENERAL

Este bloque de clases interactivas se centra en la transición de aplicaciones monolíticas a arquitecturas modernas desacopladas. Se abordará la lógica de negocio avanzada en el servidor, la creación de APIs RESTful y el consumo asíncrono desde el cliente, utilizando Docker como estándar de virtualización.

SESIÓN 1: LÓGICA DE NEGOCIO, SESIONES Y ARQUITECTURA MVC

Introducción Teórica y Conceptos Clave

La arquitectura de software permite organizar el código para su mantenimiento. El patrón **MVC (Modelo-Vista-Controlador)** separa la persistencia de datos, la lógica de negocio y la interfaz de usuario.

- **Sanitización:** Limpieza de entradas (\$_POST) para evitar XSS e Inyección SQL.
- **Sesiones:** Mecanismo para persistir el estado del usuario entre peticiones HTTP mediante PHPSESSID y Cookies HTTP-only.

Caso de Estudio: Módulo de Autenticación MVC

Router.php (Front Controller):

```
<?php
class Router {
    protected $routes = [];
    public function add($method, $uri, $controller) {
        $this->routes[] = ['method' => $method, 'uri' => $uri,
'        'action' => $controller];
    }
    public function handle($method, $uri) {
        foreach($this->routes as $route) {
            if($route['method'] == $method && $route['uri'] == $uri)
{
                return call_user_func($route['action']);
            }
        }
    }
}
```

SESIÓN 2: SERVICIOS WEB Y ARQUITECTURA DE APIS RESTFUL

Una API RESTful utiliza el protocolo HTTP para transferir representaciones de recursos (usualmente JSON). Se basa en la statelessness (sin estado) y el uso de verbos estándar.

- **GET:** Obtener recursos (200 OK).
- **POST:** Crear recursos (201 Created).
- **PUT/DELETE:** Actualizar o eliminar (200 OK / 204 No Content).

Implementación de API (api/productos.php):

```
<?php
header("Content-Type: application/json");
$method = $_SERVER['REQUEST_METHOD'];
switch($method) {
    case 'GET':
```

```

        echo json_encode(['data' => $model->getAll()]);
        break;
    case 'POST':
        $input = json_decode(file_get_contents('php://input'),
true);
        $res = $model->create($input);
        http_response_code(201);
        echo json_encode(['message' => 'Creado']);
        break;
}

```

SESIÓN 3: INTEGRACIÓN FRONTEND-BACKEND (FETCH API)

El consumo de APIs permite crear Single Page Applications (SPA) donde la página no se recarga, sino que el DOM se actualiza dinámicamente mediante peticiones asíncronas.

Consumo Asíncrono en JavaScript:

```

async function getProducts() {
    try {
        const response = await
fetch('http://localhost:8080/api/productos.php');
        const json = await response.json();
        renderTable(json.data);
    } catch (error) {
        console.error('Error en fetch:', error);
    }
}

```

Caso de Estudio: SISTESIS-UNI (Sistema Integral de Seguimiento y Gestión de Tesis Monográficas)

1. Presentación General y Arquitectura de Infraestructura

El sistema **SISTESIS-UNI** es una plataforma diseñada para automatizar el flujo de aprobación de protocolos y tesis monográficas en la UNI. Para garantizar la portabilidad y escalabilidad, el sistema se basa en una arquitectura de microservicios contenerizados utilizando Docker.

1.1 Configuración de Docker Compose

Utilizaremos un entorno compuesto por un servidor web Apache con PHP 8.2, un motor de base de datos MySQL 8.0 y una interfaz administrativa phpMyAdmin.

Archivo: docker-compose.ymlversion: '3.8'

services:

web:

build:

context: .

dockerfile: docker/php/Dockerfile

ports:

- "8080:80"

volumes:

- ./var/www/html

depends_on:

- db

networks:

- sistesis-net

db:

image: mysql:8.0

restart: always

environment:

MYSQL_DATABASE: sistesis_db

MYSQL_ROOT_PASSWORD: root_password

ports:

- "3306:3306"

volumes:

- db_data:/var/lib/mysql

- ./sql/init_sistesis.sql:/docker-entrypoint-initdb.d/init_sistesis.sql

networks:

- sistesis-net

phpmyadmin:

image: phpmyadmin/phpmyadmin

ports:

- "8081:80"

environment:

PMA_HOST: db

MYSQL_ROOT_PASSWORD: root_password

networks:

- sistesis-net

networks:

sistesis-net:

driver: bridge

volumes:

db_data:

Archivo: docker/php/DockerfileFROM php:8.2-apache

RUN docker-php-ext-install pdo pdo_mysql

RUN a2enmod rewrite

COPY --chown=www-data:www-data . /var/www/html

WORKDIR /var/www/html

1.2 Script de Base de Datos (sql/init_sistemas.sql)

CREATE DATABASE IF NOT EXISTS sistemas_db;

USE sistemas_db;

CREATE TABLE usuarios (

id INT AUTO_INCREMENT PRIMARY KEY,

nombre VARCHAR(100) NOT NULL,

correo VARCHAR(100) UNIQUE NOT NULL,

password VARCHAR(255) NOT NULL,

rol ENUM('estudiante', 'docente', 'comision') DEFAULT 'estudiante',

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

);

CREATE TABLE tesis (

id INT AUTO_INCREMENT PRIMARY KEY,

titulo VARCHAR(255) NOT NULL,

descripcion TEXT,

estado ENUM('propuesta', 'aprobado', 'rechazado', 'finalizado') DEFAULT 'propuesta',

```

    estudiante_id INT,

    FOREIGN KEY (estudiante_id) REFERENCES usuarios(id) ON DELETE CASCADE
);

CREATE TABLE revisiones (

    id INT AUTO_INCREMENT PRIMARY KEY,

    tesis_id INT,

    revisor_id INT,

    comentario TEXT,

    fecha_revision TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (tesis_id) REFERENCES tesis(id),

    FOREIGN KEY (revisor_id) REFERENCES usuarios(id)

);

-- Usuario inicial (password: admin123)

INSERT INTO usuarios (nombre, correo, password, rol)

VALUES ('Comisión Académica', 'comision@uni.edu.ni', 'FW34', 'comision');

```

2. Sesión 1: Arquitectura MVC en Backend y Sesiones Seguras

2.1 Marco Teórico

- **Ciclo Request-Response:** El cliente envía una petición HTTP, el servidor la procesa y devuelve una respuesta (HTML, JSON).
- **Sanitización:** Uso de `filter_var()` para limpiar entradas y prevenir ataques XSS e Inyecciones SQL.
- **Seguridad de Sesiones:**
 - ✓ **Session Hijacking:** Robo de la cookie de sesión. Se mitiga usando `HttpOnly` y `Secure`.
 - ✓ **Session Fixation:** Ataque donde el atacante impone un ID de sesión. Se mitiga con `session_regenerate_id(true)`.

- **Patrón MVC:** Separa la lógica de negocio (Model), la interfaz de usuario (View) y el flujo de control (Controller).

2.2 Implementación del Módulo de Autenticación

Archivo: .htaccessRewriteEngine On

RewriteCond %{REQUEST_FILENAME} !-f

RewriteCond %{REQUEST_FILENAME} !-d

RewriteRule ^(.*)\$ index.php [L,QSA]

Archivo: config/Database.php<?php

```
class Database {  
  
    private static $instance = null;  
  
    private $conn;  
  
    private function __construct() {  
  
        $dsn = "mysql:host=db;dbname=sistesis_db;charset=utf8mb4";  
  
        $this->conn = new PDO($dsn, "root", "root_password", [  
  
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,  
  
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC  
  
        ]);  
  
    }  
  
    public static function getInstance() {  
  
        if (!self::$instance) self::$instance = new Database();  
  
        return self::$instance->conn;  
  
    }  
  
}
```

Archivo: models/UserModel.php

<?php

```
require_once __DIR__ . '/../config/Database.php';
```

```
class UserModel {
```

```
    private $db;
```

```
    public function __construct() {
```

```
        $this->db = Database::getInstance();
```

```
    }
```

```
    public function getByEmail($email) {
```

```
        $stmt = $this->db->prepare("SELECT * FROM usuarios WHERE correo = ?");
```

```
        $stmt->execute([$email]);
```

```
        return $stmt->fetch();
```

```
    }
```

```
}
```

Archivo: controllers/AuthController.php

<?php

```
require_once __DIR__ . '/../models/UserModel.php';
```

```
class AuthController {
```

```
    public function login() {
```

```
        if ($_SERVER['REQUEST_METHOD'] === 'POST') {
```

```
            $email = filter_var($_POST['email'], FILTER_SANITIZE_EMAIL);
```

```
            $password = $_POST['password'];
```

```
            $userModel = new UserModel();
```

```
            $user = $userModel->getByEmail($email);
```

```

if ($user && password_verify($password, $user['password'])) {
    session_start([
        'cookie_httponly' => true,
        'cookie_secure' => false, // Cambiar a true en producción con HTTPS
        'cookie_samesite' => 'Lax'
    ]);
    session_regenerate_id(true);
    $_SESSION['user_id'] = $user['id'];
    $_SESSION['user_name'] = $user['nombre'];
    header('Location: /dashboard');
    exit();
}

$error = "Credenciales inválidas";
}

include __DIR__ . '/../views/login.php';
}

public function logout() {
    session_start();
    session_destroy();
    header('Location: /login');
}
}

```

2.3 Guía de Pruebas de Laboratorio

1. **Paso 1:** Iniciar contenedores: docker-compose up -d.
2. **Paso 2:** Acceder a <http://localhost:8080/login>.
3. **Paso 3:** Ingresar credenciales: comision@uni.edu.ni / admin123.
4. **Paso 4:** Abrir F12 > Application > Cookies.
5. **Paso 5:** Verificar que la cookie PHPSESSID tenga marcada la casilla HttpOnly.
6. **Paso 6:** Intentar acceder a /dashboard sin loguearse para verificar la protección de rutas.

2.4 Ejercicios de Estudio Independiente (Sesión 1)

1. **Seguridad:** Implementar un límite de intentos de login (3 intentos) almacenando los fallos en la base de datos.
2. **MVC:** Crear el TesisController.php y su vista correspondiente para listar las tesis del estudiante logueado.
3. **Validación:** Modificar el registro de usuarios para que la contraseña requiera al menos 8 caracteres, una mayúscula y un número.

3. Sesión 2: Arquitectura de APIs RESTful y Postman

3.1 Marco Teórico

- **REST vs SOAP:** REST es ligero, utiliza verbos HTTP y JSON, ideal para aplicaciones web modernas.
- **Verbos HTTP:** GET (Leer), POST (Crear), PUT (Actualizar), DELETE (Eliminar).
- **Códigos de Estado:** 201 (Created), 400 (Bad Request), 404 (Not Found), 500 (Server Error).

3.2 Implementación de la API de Tesis

Archivo: models/TesisModel.php

```
<?php
class TesisModel {
    private $db;
    public function __construct() { $this->db = Database::getInstance(); }

    public function getAll() {
        return $this->db->query("SELECT * FROM tesis")->fetchAll();
    }
    public function create($data) {

        $stmt = $this->db->prepare("INSERT INTO tesis (titulo, descripcion, estudiante_id)
VALUES (?, ?, ?)");
```

```

        return $stmt->execute([$data['titulo'], $data['descripcion'], $data['estudiante_id']]);
    }

    public function update($id, $estado) {

        $stmt = $this->db->prepare("UPDATE tesis SET estado = ? WHERE id = ?");

        return $stmt->execute([$estado, $id])
        ;
    }
}

```

****Archivo: `api/tesis.php` (Endpoint)****

```
```php
```

```
<?php
```

```
header("Content-Type: application/json");
```

```
require_once __DIR__ . '/../models/TesisModel.php';
```

```
$model = new TesisModel();
```

```
$method = $_SERVER['REQUEST_METHOD'];
```

```
switch($method) {
```

```
 case 'GET':
```

```
 echo json_encode($model->getAll());
```

```
 break;
```

```
 case 'POST':
```

```
 $data = json_decode(file_get_contents("php://input"), true);
```

```
 if($model->create($data)) {
```

```
 http_response_code(201);
```

```
 echo json_encode(["message" => "Tesis creada"]);
```

```

 }

 break;
}

```

### 3.3 Guía de Pruebas con Postman

1. **Paso 1:** Crear una nueva Request tipo **GET** a `http://localhost:8080/api/tesis.php`.
2. **Paso 2:** Crear una Request tipo **POST** a la misma URL.
3. **Paso 3:** En la pestaña **Body**, seleccionar **raw** y formato **JSON**.
4. **Paso 4:** Enviar el JSON: `{"titulo": "IA en Medicina", "descripcion": "Uso de redes neuronales", "estudiante_id": 1}`.
5. **Paso 5:** Verificar el Status Code 201 Created.

## 4. Sesión 3: Frontend SPA y Fetch API

### 4.1 Marco Teórico

- **Asincronía:** Capacidad de ejecutar procesos en segundo plano sin bloquear el hilo principal.
- **Fetch API:** Interfaz moderna para realizar peticiones HTTP asíncronas.
- **SPA (Single Page Application):** Aplicaciones que cargan una sola página y actualizan el contenido dinámicamente.

### 4.2 Cliente Frontend SISTESIS-UNI

**Archivo:** `public/js/tesis_client.js`

```

async function cargarTesis() {

```

```

 const res = await fetch('/api/tesis.php');

 const data = await res.json();

 const tabla = document.getElementById('tabla-tesis');

 tabla.innerHTML = data.map(t => `

 <tr>

 <td class="p-2 border">${t.titulo}</td>

 <td class="p-2 border">${t.estado}</td>

 <td class="p-2 border">

```

```
<button onclick="cambiarEstado(${t.id}, 'aprobado')" class="bg-blue-500 text-white p-1">Aprobar</button>
```

```
</td>
```

```
</tr>
```

```
`).join(");
```

```
}
```

```
document.addEventListener('DOMContentLoaded', cargarTesis);
```

### 4.3 Guía de Laboratorio

1. **Paso 1:** Abrir public/app\_tesis.html.
2. **Paso 2:** Observar que la tabla está vacía al inicio.
3. **Paso 3:** Abrir Chrome DevTools (F12) > Network.
4. **Paso 4:** Recargar la página y ver la petición XHR/Fetch a tesis.php.
5. **Paso 5:** Usar el formulario para agregar una tesis y observar cómo se actualiza la tabla sin recargar el navegador.

### 5. Rúbrica de Evaluación de Desempeño

| Criterio                     | Excelente (25%)                                                  | Bueno (15%)                                           | Necesita Mejora (5%)                             |
|------------------------------|------------------------------------------------------------------|-------------------------------------------------------|--------------------------------------------------|
| <b>Arquitectura MVC</b>      | Estructura perfecta de carpetas y uso correcto de Singleton PDO. | Estructura MVC funcional pero con lógica mezclada.    | No separa responsabilidades de Model/Controller. |
| <b>Seguridad de Sesiones</b> | Implementa regeneración de ID y banderas HttpOnly.               | Usa sesiones pero sin medidas de seguridad avanzadas. | Las sesiones son vulnerables a secuestro básico. |
| <b>Implementación API</b>    | CRUD completo siguiendo estándares REST y códigos HTTP.          | API funcional pero no usa verbos HTTP correctamente.  | Endpoint confuso o no devuelve JSON válido.      |
| <b>Frontend SPA</b>          | Uso fluido de Async/Await y manipulación dinámica del DOM.       | Usa Fetch pero tiene errores de manejo de promesas.   | Recarga la página para mostrar nuevos datos.     |